

Fast-Test Homework

This task is about the fast-test approach (TDD, BDD or other names.. :))

Read the Appendix for coding guidelines. Feel free to send us questions about the task.

There is no focus on UI on this task.

Part 1

In this part you are required to refactor a “simple” React into a fast-test approach. The focus here is on “flat presentation”, or completely separating the UI from the logic.

Intro

We have a page that shows a list of “books”, plus a button to add a book (some mocked hardcoded data).

From a technical point of view, there is API that uses the format

[https://tdd.demo.reaktivat.com/v1/books/\[user\]](https://tdd.demo.reaktivat.com/v1/books/[user])

Where “user” is your identifier (nickname).

API swagger: <https://tdd.demo.reaktivat.com/api-docs/>

NB: API has SSL self-signed, so to work you’ll need to open any API endpoint in the browser directly and “allow” this certificate to be used.

The Task

Please **refactor** the existing solution:

<https://codesandbox.io/s/reaktivat-tdd-challenge-1-sample-begin-ljipo>

* To save your own version:

- Login to codesandbox
- Use “fork” and save your own version

Note: You don’t have to complete the task within the codesandbox. You may use any other way to share the result. It is important to have a clear guidance on how to run the code.

Requirements for refactoring

The Basic requirement is to have the **logic separated from the rendering**, so that it’s possible to have **test-on-logic**. Logic means business, and the majority of UX.

Acceptance Criteria:

1. **Zero logic and calculations in TSX/JSX files** – Use MVP (or, in other name, MVVM) pattern where the component transfers all the logic to the Controller. It’s Important to separate the View, and be able to test. The rest is less important. Any JS code besides connecting the Controller in View (JSX) is actually incorrect.

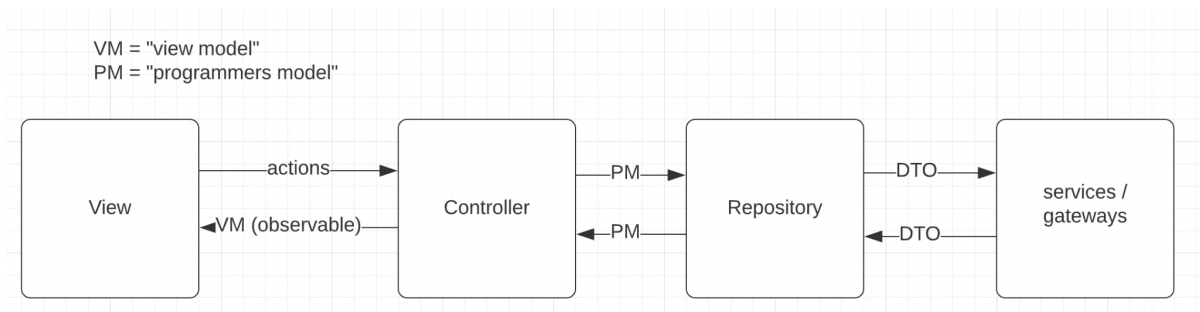
2. Use MobX to hold state and mobx-react for reactivity. In case you don't know MobX – come up with your own solution to MVVM implementation.
3. Implement books creation
4. Cover logic with tests

Part 2 – Power-up task

- a. The customer is requesting a “switch” on the interface, between “All books” and “Private books” (see API for reference).
These two are mutually exclusive (buttons or radio or tabs, not checkboxes).
- b. Also, one more sticky **application-wide** header is requested. The header should have a counter with a private books number, like “Your books: 30”.

Refactoring target: Architecture pattern

Naming is taken from the following scheme:



That's more or less a MVP (model-view-presenter) pattern. “Repository” layer is optional if you don't see a use of it, it's fine to use the service directly.

APPENDIX:

CodeStyle

Note: style guidelines are given in assumption MobX is used for reactivity. If another state-management library is used - some of the bullets are not viable.

1. Files structure must account for splitting of the reusable components and feature-specific components.
2. Every component that **has logic** (any branching is a logic) or **requires an access to services/gateways**, has its own controller for that purpose.
3. If needed – use a global controller or store that only contains data which could be shared across the whole application e.g. userStore, uiStore etc.

4. In View, there is only Mobx stores usage. All the logic should be in Controllers, View is only for rendering.
5. Test logic is a must. Snapshot- and property- testing is optional
6. Views are functional
7. For local controllers - care about lifecycle and resources disposal
8. Make sure we don't have unnecessary renders caused by MobX reactivity (see `runInAction` doc function for details)
9. Remove `console.log`, use `console.debug` in case if you need to leave it for debug purpose